

Pit Wall Predictor

Interview report - real-data Formula 1 strategy analysis system

A transparent offline Python project that turns completed-race CSV files into engineering-style race pace, tyre, strategy, and reporting outputs.

Project snapshot	Value
Purpose	Post-race Formula 1 strategy analysis with transparent formulas and reusable CLI, GUI, and browser front ends.
Runtime	Python, NumPy, Pandas, SciPy, Matplotlib, Tkinter, and the standard library.
Verified data	8 completed races, 9,215 laps, 22 drivers, 11 teams, imported from local official race CSV files.
Outputs	CSV summaries, Markdown reports, PDF reports, PNG plots, and HTML replays.
Modes	CLI, colourful Tkinter dashboard, and localhost browser dashboard.

Verified on 2026-07-01, the current imported bundle contains 8 completed races and 9215 lap rows. The analysis layer now loads that real bundle automatically whenever it is present.

The main design goal is clarity: every score, plot, and replay is generated from explicit rules that can be explained in a technical interview without relying on black-box machine learning.

1. What the project is

Pit Wall Predictor is a post-race analysis tool, not a live prediction engine. It reads completed-race lap CSV files, cleans the laps, compares drivers, scores race execution, fits a simple tyre-degradation curve, and then exposes the same backend through three surfaces: command line, Tkinter desktop UI, and a local web UI.

That makes the project useful as a portfolio piece because it shows data loading, data cleaning, transparent scoring, plotting, animation, local web serving, and report generation in one coherent codebase.

2. How to use it

Mode / command	What it does	When to use it
<code>python main.py</code>	Runs the default CLI analysis for the default race and driver pair.	Fastest way to smoke-test the backend.
<code>python main.py --race "Monaco Grand Prix" --driver VER --compare NOR</code>	Chooses a race plus two driver codes.	When you want a specific post-race comparison.
<code>python main.py --gui</code>	Launches the colourful Tkinter dashboard.	When you want a desktop front end.
<code>python main.py --web</code>	Starts the localhost browser interface.	When you want the more polished web UI.
<code>python main.py --animations</code>	Exports the HTML replay animations.	When you need replay artefacts for a presentation.

3. What you can input

Input	Example	Where it is used
Race name	Barcelona-Catalunya Grand Prix	Selects the completed race from the calendar.
Primary driver code	HAM	Drives the main analysis summary and tyre fit.
Comparison driver code	RUS	Builds the driver-battle verdict and pace plot.
Season	2026	Matches the calendar and imported data bundle.
UI flag	<code>--gui</code> or <code>--web</code>	Chooses the desktop or browser surface.
Replay flag	<code>--animations</code>	Creates the HTML replay exports.
Real-data manifest	<code>data/real/source_manifest.csv</code>	Tells the importer where the official CSV files live.

4. What comes out

Output family	Example file or surface	Why it matters
Grid summary	<code>outputs/reports/*_all_driver_summary.csv</code>	Gives one row per driver with pace, consistency, tyre, and total race-engineer score.
Readable report	<code>outputs/reports/*_post_race_report.md</code> and <code>.pdf</code>	Turns the analysis into a portfolio-friendly narrative.
Dashboard plots	<code>outputs/plots/*.png</code>	Provides the visual evidence behind the scores.
Replays	<code>outputs/animations/*.html</code>	Shows the race, tyre, and pit-stop stories in motion.

Output family	Example file or surface	Why it matters
Browser UI	<code>http://127.0.0.1:8000</code>	Lets a user explore the project in a local web interface.
Tkinter UI	<code>main.py --gui</code>	Provides the desktop variant without a browser.

5. How the analysis works

5.1 Data loading and validation

The loader prefers the imported real-data bundle in `data/real/` whenever it exists. If that bundle is missing, the project falls back to the deterministic sample dataset so the app still opens during development. The import pipeline stores the bundle in a local calendar file, a consolidated lap file, a manifest copy, and a source index that tracks the original CSV source for each race.

The current validated bundle contains 8 completed races, 22 drivers across the bundle, and 9215 lap rows. The verified completed rounds are: Australian Grand Prix, Chinese Grand Prix, Japanese Grand Prix, Miami Grand Prix, Canadian Grand Prix, Monaco Grand Prix, Barcelona-Catalunya Grand Prix, Austrian Grand Prix.

5.2 Clean-lap filtering

Clean pace is the foundation of the project. The cleaning rule removes pit laps, safety-car laps, in-laps, out-laps, and then drops any remaining lap that is more than 4.0 seconds slower than the driver's median clean lap. That keeps the pace comparison focused on representative racing pace rather than traffic, pit cycles, or extreme anomalies.

5.3 Driver analysis

For each driver, the backend calculates average clean pace, fastest clean lap, slowest clean lap, standard deviation, pit-stop count, compound usage, and stint-level summaries. Each stint also gets a simple linear degradation estimate so the report can say where pace held up and where it dropped away.

5.4 Scoring model

The gamified score is intentionally transparent. It combines pace, consistency, tyre management, and stint execution with fixed weights, then converts the result into an easy-to-read rating and badge. Because the weights are explicit, the interview story is straightforward: the score is an explanation layer, not a hidden AI model.

5.5 Tyre degradation model

The tyre model uses SciPy's `curve_fit` on clean laps from a single driver and compound. The fitted form is $\text{Lap Time} = \text{Base Pace} + a \times \text{Tyre Age} + b \times \text{Tyre Age}^2$. The project reports the fitted coefficients and the RMSE so a reader can see both the trend and the quality of the fit.

6. Formula cheat sheet

Metric	Formula or rule	Interpretation
Clean pace	Mean of clean laps after filtering	Core pace metric for the driver and the grid.
Consistency score	<code>clip(100 - lap_time_std x 30)</code>	Lower lap-time scatter is rewarded.

Metric	Formula or rule	Interpretation
Tyre management score	$\text{clip}(100 - \text{avg_positive_degradation} \times 430)$	Punishes steep stint degradation.
Pace score	$\text{clip}(100 - \text{pace_deficit} \times 22)$	Converts clean pace deficit into a 0-100 scale.
Stint execution score	$\text{clip}(100 - \text{stint_pace_spread} \times 18)$	Rewards a balanced stint profile.
Race engineer score	$0.35 \text{ pace} + 0.25 \text{ consistency} + 0.25 \text{ tyre} + 0.15 \text{ stint}$	Final gamified score used in the ranking.
Badge logic	Threshold checks on pace, consistency, tyres, stints, and position	Produces a human-readable achievement label.

The key design decision is that every formula is simple enough to explain live in an interview. There is no opaque model tuning step and no hidden training set - just fixed rules, visible assumptions, and stable outputs.

7. Example output from the verified Barcelona-Catalunya run

Rank	Driver	Team	Clean pace (s)	Score	Badge
1	RUS	Mercedes	82.369	75.8	Clean Air King
2	ANT	Mercedes	82.303	75.0	Clean Air King
3	HAM	Ferrari	81.642	73.7	Race Pace Beast
4	NOR	McLaren	82.381	73.3	Clean Air King
5	VER	Red Bull Racing	82.225	69.6	Tyre Cliff Victim

On this verified run, Hamilton leads the sample summary and the report generator turns that result into a compact project artefact. That makes the project easy to demo: the same backend can support a short summary, a detailed explanation, and a visual dashboard from the same race data.

8. Plot and replay catalogue

Artifact	What it shows
full_grid_pace_ranking.png	Average clean pace by driver; lower is faster.
full_grid_consistency_ranking.png	Lap-time stability across the grid.
full_grid_tyre_management_ranking.png	Who preserved tyre life best.
team_pace_comparison.png	Average pace by team.
driver_battle_HAM_vs_RUS.png	Lap-by-lap clean pace comparison.
full_grid_strategy_timeline.png	Tyre stints and pit laps across the whole field.
race_pace_heatmap.png	Relative clean-lap pace trace by driver and lap.
tyre_degradation_HAM_M.png	SciPy tyre-age curve fit for the selected driver.
race_pace_replay.html	Animated race-progress replay.
tyre_degradation_replay_HAM_M.html	Animated tyre-fit reveal.
pit_stop_timeline_replay.html	Animated stint and pit-stop timeline.

9. Real-data import pipeline and verification

The project now reads local FastF1-style CSV exports through a manifest-driven importer. The manifest preserves the source path for each completed race, the importer normalizes the lap schema into the project's standard columns, and the loaded bundle is then used by the CLI, GUI, browser UI, and report generation.

Round	Race	Laps in bundle	Drivers	Status
1	Australian Grand Prix	1000	20	completed
2	Chinese Grand Prix	904	18	completed
3	Japanese Grand Prix	1086	22	completed
4	Miami Grand Prix	1033	22	completed
5	Canadian Grand Prix	1206	21	completed
6	Monaco Grand Prix	1415	21	completed
7	Barcelona-Catalunya Grand Prix	1233	22	completed
8	Austrian Grand Prix	1338	22	completed

The validation pass confirmed that all eight supplied completed rounds are visible to the app. That matters for an interview because it shows the project is not only functional in isolation, but also wired to a source-tracked, reproducible data path.

10. Project structure

Module	Responsibility	Notes
main.py	CLI entry point for demo, GUI, web, and animation modes.	Routes to the right surface.
src/real_data_importer.py	Imports and normalizes real race CSV files.	Tracks source files and builds the local bundle.
src/data_loader.py	Loads real or sample data depending on what is present.	Keeps the rest of the app simple.
src/data_cleaning.py	Applies the clean-lap rules.	Removes pits, safety-car laps, and outliers.
src/driver_analysis.py	Builds per-driver and full-grid analysis.	Produces the summary table.
src/scoring.py	Applies the visible scoring and badge rules.	Turns metrics into a gamified rating.
src/tyre_degradation.py	Fits the quadratic tyre-age model.	Uses SciPy curve fitting and RMSE.
src/visualizer.py	Creates the PNG dashboard plots.	Matplotlib output only.
src/animations.py	Creates HTML replays and static fallbacks.	Kept dependency-light.
src/web_app.py	Serves the local browser dashboard.	Reuses the same backend calculations.
src/gui.py	Tkinter desktop dashboard.	Alternate offline front end.
src/report_generator.py	Writes CSV, Markdown, and PDF report artefacts.	Used for portfolio exports.

11. Interview talking points

If you want a crisp spoken summary, describe the project as an offline Formula 1 post-race engineering dashboard that turns source-tracked lap CSVs into pace, consistency, tyre, strategy, and replay outputs. Emphasize that the value is in transparency: every number comes from a visible rule or model, and the same backend powers the CLI, GUI, browser UI, plots, animations, and report export.

That framing makes it easy to talk about software engineering, data handling, analysis, and product thinking in one story, which is exactly what an interview panel usually wants to hear.